

EPIC 05 — Inter-Service Communication & Resilience

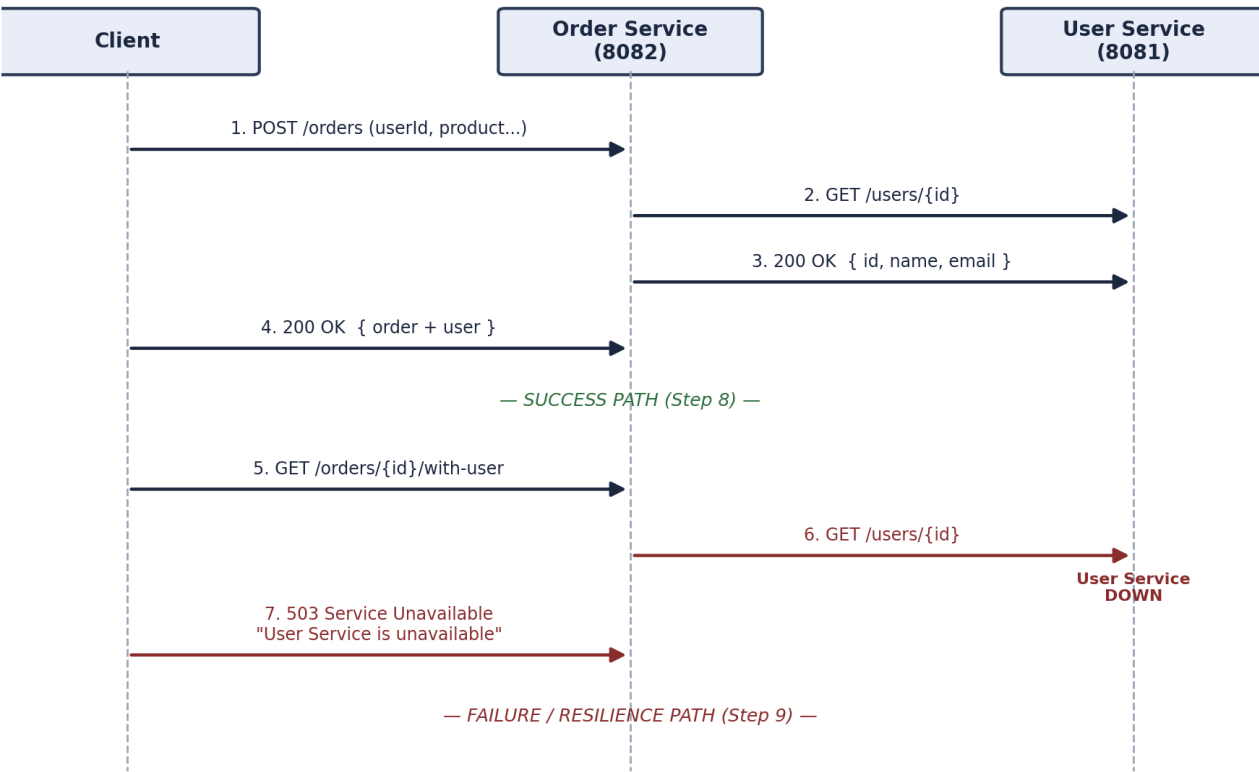
JIRA Task 01 · REST-Based Inter-Service Communication · Sprint 12 (21/9 – 4/10)

Objective

Build two Spring Boot microservices — **User Service** and **Order Service** — that communicate over REST. Order Service must never access the User Service database directly; it must call User Service's own API whenever it needs user information. The task also covers returning a combined response and handling the case where User Service is unreachable (resilience).

Service	Port	Role
User Service	8081	Owns user data. Exposes GET /users and GET /users/{id}
Order Service	8082	Owns order data. Calls User Service before creating/returning an order

Sequence Diagram



The green path (1–4) is the normal success flow: Order Service asks User Service for the user, gets back the details, and returns a combined response to the client. The red path (5–7) is the resilience test: when User Service is down, Order Service does not crash — it catches the failure and returns a clean 503 response instead.

Step-by-Step: What Was Built

Step 1–2 — Understand & set up the two services

Learned the concepts (sync vs async communication, REST, request-response model). Created two separate Spring Boot projects — **user-service** (port 8081) and **order-service** (port 8082) — each with its own MySQL database.

Step 3 — User API

User Service exposes GET /users and GET /users/{id}, returning id, name and email.

Step 4 — Order API

Order Service exposes create, get-by-id, get-by-user, update-status and delete endpoints for orders. Each order stores userId, productName, quantity, amount and status.

Step 5–6 — Service-to-service call

Added a RestTemplate bean (Spring's HTTP client) and a UserServiceClient class in Order Service. Before saving a new order, Order Service calls User Service's GET /users/{id} to confirm the user actually exists. If not, the order is rejected with a clean 400 response instead of a raw database write.

Step 7 — Combined response

Added a new endpoint, GET /orders/{id}/with-user, that fetches the order and the full user record together and returns them nested in a single JSON response — order details plus a nested user object.

Step 8 — Test successful communication

With both services running: created an order for a real userId (200 OK), then called GET /orders/{id}/with-user and confirmed the response contained both the order and the nested user info.

Step 9 — Test failure / resilience

Stopped User Service, then called the same endpoint again. Instead of crashing with a raw 500 error, Order Service caught the connection failure and returned 503 Service Unavailable with the message "User Service is unavailable. Please try again later."

How To Run & Test It Yourself

- 1 Make sure MySQL is running, and both `user_db` and `order_db` databases exist.
- 2 Start **User Service** first (VS Code Run button on the main class, or `mvnw.cmd spring-boot:run` from its folder). Confirm the terminal shows Tomcat started on port(s): 8081.
- 3 Start **Order Service** the same way. Confirm Tomcat started on port(s): 8082.
- 4 In Postman, create a user or confirm one exists: GET `http://localhost:8081/users`.
- 5 Create an order with a real `userId`: POST `http://localhost:8082/api/v1/orders` — expect 200 OK.
- 6 Fetch the combined view: GET `http://localhost:8082/api/v1/orders/{id}/with-user` — expect the order with a nested user object.
- 7 Stop User Service, repeat the same request — expect 503 Service Unavailable with a clear message, not a crash.
- 8 Restart User Service and repeat once more to confirm it recovers back to 200 OK.

Tip: if a service won't start with "Port already in use", an old instance from a previous run is likely still active. Find it with `netstat -ano | findstr :PORT` and stop it with `taskkill /PID <pid> /F` before restarting.

Deliverables Checklist

- User Service — running, GET `/users` and GET `/users/{id}` working
- Order Service — running, full CRUD for orders working
- REST service-to-service communication — Order Service calls User Service via RestTemplate
- Combined API response — GET `/orders/{id}/with-user` returns order + nested user
- Postman collection — requests saved for create order, get order, with-user, failure case
- Sequence diagram — this document
- Documentation — this document
- Git commit — push the final code to the repository